Osvrt na Zadatak 2 — CDCL i proširenja

Bilješke nakon implementacije CDCL solver-a. Provjera plana iz `zadatak1-osvrt.md` i pojam što se *stvarno* promijenilo u odnosu na čisti DPLL.

Što sam najavio i što sam doista isporučio

Iz osvrta nakon Z1 sam planirao **četiri značajna proširenja**:

Plan iz Z1	Status u Z2
CDCL — 1UIP učenje klauzula	✅ `CdclSolver.analyze()`
Graf implikacija + nekronološki povratak (backjumping)	✅ `reason[v]`, `level[v]` + `cancelUntil(backLvl)`
Two-watched literals (Chaff)	✅ `propagate()` + `posWatches`/`negWatches`
Restart s Luby nizom	✅ `luby()` + glavna petlja

Plus uz CDCL **VSIDS** + **phase saving** (ne računa se kao zasebno proširenje, ali je standardno).

Što se konkretno promijenilo u kodu

1) Struktura solver-a — rekurzija → iteracija

`DpllSolver` je **rekurzivan** — klasični "try `l`, ako ne uspije try `¬l`":

```
boolean dpll() {
    if (simplify() == CONFLICT) return false;
    if (allAssigned())          return true;
    int lit = chooseBranchLiteral();
    int mark = trailSize;
    assignLit(lit);
    if (dpll()) return true;
    backtrackTo(mark);
    assignLit(-lit);
    return dpll();
}
```

`CdclSolver` je **iterativan** — jedna velika `while(true)` petlja koja naizmjenično propagira i analizira konflikte:

```
while (true) {
    int[] confl = propagate();
    if (confl != null) {
        if (currentDL() == 0) return UNSAT;
        (learnt, backLvl) = analyze(confl);
```

```
            cancelUntil(backLvl);
            add learnt; enqueue assertingLiteral;
        } else {
            if (allAssigned()) return SAT;
            if (restartTime()) cancelUntil(0);
            int v = pickVSIDS();
            newDecisionLevel();
            enqueue(decision);
        }
    }
```

To je veliki strukturni rez — backjump preko više razina **odjednom** ne ide "prirodno" kroz rekurziju jer bi morao baciti više `return false`-a uzastopno.

2) Reprezentacija formule

	DpllSolver (Z1)	**CdclSolver (Z2)**
Klauzule	`List<int[]>` (fiksne, iz inputa)	`List<int[]>` originalne + dinamička lista naučenih
Propagacija	Linearna petlja po svim klauzulama	Watched literals: `posWatches[v]` / `negWatches[v]`
Dodjela	`value[v]` + `int[] trail`	+ `level[v]`, `reason[v]` (graf implikacija)
Razine odluke	implicitno kroz rekurziju	eksplicitan `int[] trailLim`
Heuristika	DLIS — broji pojavljivanja kroz cijelu formulu	VSIDS — `double[] activity` + bump/decay

3) Najveći "Aha!" moment — invariant praćenih literala

Pri implementaciji two-watched literals trebalo mi je da shvatim:

> Klauzula je "u redu" sve dok je *barem jedan* od dva praćena literala `true` **ili** *barem dva* nepridijeljena. Treba je gledati tek kad *oba* praćena postanu `false`.

Posljedica je da pri propagaciji literala `p` ne prolazimo kroz sve klauzule nego **samo kroz `watches[-p]`** (one koje su praćenu vrijednost `-p` koristile kao watch). To je glavni razlog što CDCL preživi na `hanoi4` u 78 ms umjesto 5+ minuta.

4) Tihi heroj — propagacija s reason klauzulom

Mali ali važan detalj: kad watched-literals propagacija forsira literal `l`, nazovem `enqueue(l, reasonClause)` i u `reason[abs(l)]` spremam *baš tu klauzulu koja ga je natjerala*. To `analyze()` koristi za prolaz unatrag kroz graf implikacija. Ako nema reason-a (npr. decision literal), trajno je `null` — to je signal "ovo je odluka, ne implikacija".

Mjerenja — predviđanja vs. stvarnost

Predviđanje iz Z1 osvrta

> "ponovno istraživanje istih konfliktnih grana s drugačijim okolnostima → rješenje: CDCL + 1UIP klauzule"

Stvarnost

Problem	DPLL	CDCL	Faktor
`hanoi4`	TIMEOUT >5 min	78 ms	**>3800×**
`bf0432-007`	TIMEOUT >60 s	22 ms	**>2700×**
`uf20-01`	8 ms	8 ms	~1×
`medium`	16 ms	8 ms	2×
`hole7`	66 ms	178 ms	**0.37× (sporiji!)**

Predviđanje za teške probleme pogođeno potpuno — i to s velikom rezervom. **Stvarnost je čak bolja od očekivane** (predviđao sam možda 10×, dobio 3800×).

Iznenađenje: **`hole7` je s CDCL-om sporiji**. Pigeonhole je eksponencijalan za sve DPLL/CDCL varijante bez specijalne podrške (simetrije, kardinalne klauzule). DLIS na 56 varijabli slučajno dobro pogađa, dok CDCL "investira" u učenje klauzula koje se ionako moraju eksponencijalno množiti.

Ablacija — pravi test razumijevanja

Najobjavnija stvar je bila ablacijska analiza. Isključio sam pojedina proširenja, jedno po jedno, na tri najteža problema:

Konfig	bf0432	hanoi4	hole7
svi ON	22 ms	78 ms	182 ms
`--no-cdcl`	TIMEOUT	TIMEOUT	TIMEOUT
`--no-backjump`	37 ms	105 ms	189 ms
`--no-restart`	21 ms	146 ms	67 ms
`--no-vsids`	**2717 ms**	86 ms	15 ms

Tri nezgodne istine koje me natjerala da prepoznam:

1. **CDCL je *jedini* pravi razlog što rješavač uopće radi na teškim instancama.** Bez učenja klauzula sve troje TIMEOUT. Watched literals, restart, VSIDS — sve to bez CDCL-a nema gdje napredovati.
2. **VSIDS na pigeonhole je *štetan*** — bez njega `hole7` ide u 15 ms umjesto 182 ms. Razlog: VSIDS gradi aktivnost preko konflikta, a u pigeonholeu su sve varijable simetrične, pa "aktivnost" zapravo ne nosi informaciju, samo uvodi šum + overhead održavanja.
3. **Restart ponekad odmaže.** Na `hole7` je `--no-restart` brže (67 ms vs 182 ms). Logično: ako rješavač *baš* napreduje prema dokazu UNSAT, restart ga vraća unatrag. Luby pomaže u prosjeku,

ne uvijek.

Što nije išlo glatko (lekcija o detaljima)

Bug u `--no-backjump`

Prva iteracija `--no-backjump` načina rada davala je *UNSAT na `hanoi4`* — to je formula koja je SAT. Trošio sam neko vrijeme da nađem zašto.

Uzrok: kad `analyze()` proizvede naučenu klauzulu **duljine 1** (UIP je na razini 0, dakle level-0 činjenica), upisivao sam je u trail s `reason = null` na trenutnoj decision razini umjesto na razini 0. Kasniji `cancelUntil()` ju je brisao i činjenica je *zaboravljena*.

Popravak je trivijalan u kodu:

```
if (learnt != null && learnt.length == 1) {
    targetLevel = 0;     // override: činjenica uvijek ide na level 0
} else if (config.backjump) {
    targetLevel = backLvl;
} else {
    targetLevel = Math.max(0, currentDL() - 1);
}
```

Lekcija: **level-0 činjenice nisu samo "obične implikacije" — one moraju trajno preživjeti backtrack.** U punom CDCL-u to se podrazumijeva jer backLvl za length-1 klauzulu *jest* 0; tek kad sam pokušao chronological hibrid, ova razlika je isplivala.

Što inače ne ide bez backjump-a

`--no-backjump` + CDCL je nestandardna kombinacija (Nadel-Ryvchin 2018 pokazuju da je "chronological CDCL" zapravo netrivijalan). U mojoj implementaciji radi *korektno* (provjereno: svi rezultati matchaju očekivane), ali je sporiji jer skače razinu po razinu kroz "neradne" nivoe. To je dovoljno za demo na kolokviju.

Što sam stvarno naučio

1. **CDCL = "naučiti zašto je nešto loše".** Ne pamtimo gdje smo bili, pamtimo *pravilo* koje ćemo koristiti od sada nadalje. Otud eksponencijalno bolji performans na strukturiranim problemima.
2. **Watched literals nije optimizacija — to je drugačiji algoritam.** Naivna PBO je O(klauzula × literala) po jednom literalu. Watched je amortizirano O(broj klauzula koje stvarno postanu unit). Razlika je nekoliko redova veličine.
3. **Nijedno proširenje nije univerzalno korisno.** Ablacija je to nemilosrdno pokazala — VSIDS može usporiti, restart može usporiti. Pravo umijeće je u *kompoziciji* — i zato moderni SAT solveri imaju heuristike koje *prilagođavaju* parametre u toku izvođenja (npr. SAT vs. UNSAT mode u Glucoseu).
4. **Implementacija je nekoliko stotina redaka — algoritam je *desetljeća* istraživanja.** Pravim CDCL solver-om iz 90-ih (DPLL + 1UIP + 2WL + VSIDS + restart) bi se i danas riješio velik dio SAT

competition problema. Sve poslije toga (LBM glue, chronological BT, SLS hibridi, neuralne heuristike…) je iznad ovog praga.

Mali ostatak (mogući daljnji koraci, ne traži se za bodove)

- *Scheduled lazy clause deletion* (kontrola memorijskog rasta naučenih klauzula — kod nas raste do ~6000, što je za naše benchmarkove OK)
- Indeksirani binarni heap za VSIDS umjesto linearnog skeniranja (`O(n)` po grani → `O(log n)`)
- Phase saving s polaritetnom prednošću po backbone-u
- Preprocessing (BVE — bounded variable elimination)

Sve ovo bi se uklopilo u `CdclSolver` bez velikih strukturnih rezova zahvaljujući trail/watch arhitekturi koju smo postavili u Z2.